Introduction to Software Engineering

# Project Proposal

Software Engineering Department
Faculty of Information and Technology
University of Science

# Table of Contents

Objectives 1
1 Member Contribution Assessment 2
2 Preliminary Problem Statement 3
3 Proposed Solution 5
4 Development Plan 11
5 Human Resources & Costing Plan 14
6 Tools setup 15

# Project Proposal

## Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal document with the following sections:
  - Preliminary Problem Statement
  - Proposed Solution
  - Development Plan
  - Human Resources & Costing Plan
- ✔ Understanding the Project Proposal document.

# 1 Member Contribution Assessment

| ID | Name | Role | Contribution (%) | Signature |
|---|---|---|---|---|
| 23127061 | Trương Lý Khải | **Project Manager**<br>Business Analyst<br>Tester | 100% | |
| 23127207 | Đặng Đăng Khoa | Designer<br>Backend Developer | 100% | |
| 23127271 | Võ Ngọc Bích Trâm | Designer<br>Frontend Developer<br>Tester | 100% | |
| 23127404 | Lê Tuấn Lộc | Fullstack Developer & Engineer | 100% | |
| 23127486 | Phan Quốc Thịnh | Designer<br>Frontend Developer | 100% | |

# 2 Preliminary Problem Statement

- When playing any Pokémon main series games (e.g., Pokémon Black, Pokémon Sword), players aiming to finish the whole game often need to find specific Pokémons to complete their Pokédex. They must currently search through multiple, ad-filled, and often slow-to-load wikis or complicated and overbearing guides just to find which route or area the Pokémon appears in. Our team's goal is to create a single, fast, and simple web application to allow players to build a community and search for Pokémons effectively and efficiently.

- For the operating environment, the website will be implemented using ReactJS and Tailwind CSS for user interface, NodeJS with Express framework for controlling website logic and player data. Hosting the website is done through the local server, supporting HTML5. The main programming language throughout the project is JavaScript.

- Upon launching the website, players will login to their accounts and customize their username and password to become a user. On the main page, users can view others' posts and follow other users to see their posts more frequently. Users can also discuss Pokémons and their games under any posts through a like feature and a comment system. Posts can be customized to allow for both text and images.

- Along with users, there will also be moderators employed by the main website's hosts. Moderators can warn other users about their posts or delete them if they feel that the posts are inappropriate. Moderators are differentiated from users through special highlighted tags on their profiles. If a post is deleted or a warning is issued, the user will be informed the next time they load the website.

- Along with the dashboard, players can also access the map. When using the map, players can select their desired Pokémon game. By default, the website will display the total map of the most recent release of the Pokémon IP (Pokémon Z-A as of 2025). There is also a game selector for the player should they want to select another game to track.

- Each map can be zoomed in or zoomed out depending on the selected location. A map will also display the name of each specific region of the map that is named in the game using layered text. A detailed list of every region in a game is shown below the game selector, players can scroll and select a specific region from there. Choosing a region will move the current view of the map to the center of the region.

- Near the region list will be a table showing every type of Pokémon available within the game. Pokémons are categorized either by type (Grass, Electric, ...) or by rarity (Rare, Legendary,...). There will also be a search bar for the players to search Pokémon names. Players can tag any Pokémon that they want to track. The map will display all possible locations that a tagged Pokémon can appear using a pin containing the corresponding Pokémon's icon. For each pin, if a player has explored and exhausted the corresponding area, they can mark the pin (once the area can spawn Pokémons again, the player can unmark marked pins). Hovering over each pin will also display the method of engaging with the Pokémon in that area. The location of each Pokémon is fetched from the public PokéAPI.

- Game and region selection, selected Pokémons, tagged pins and current map state are data stored specifically to each user for the map while usernames, passwords, profiles, posts, comments and likes history are stored using MongoDB.

# 3 Proposed Solution

## 3.1 Software

### 3.1.1. Features

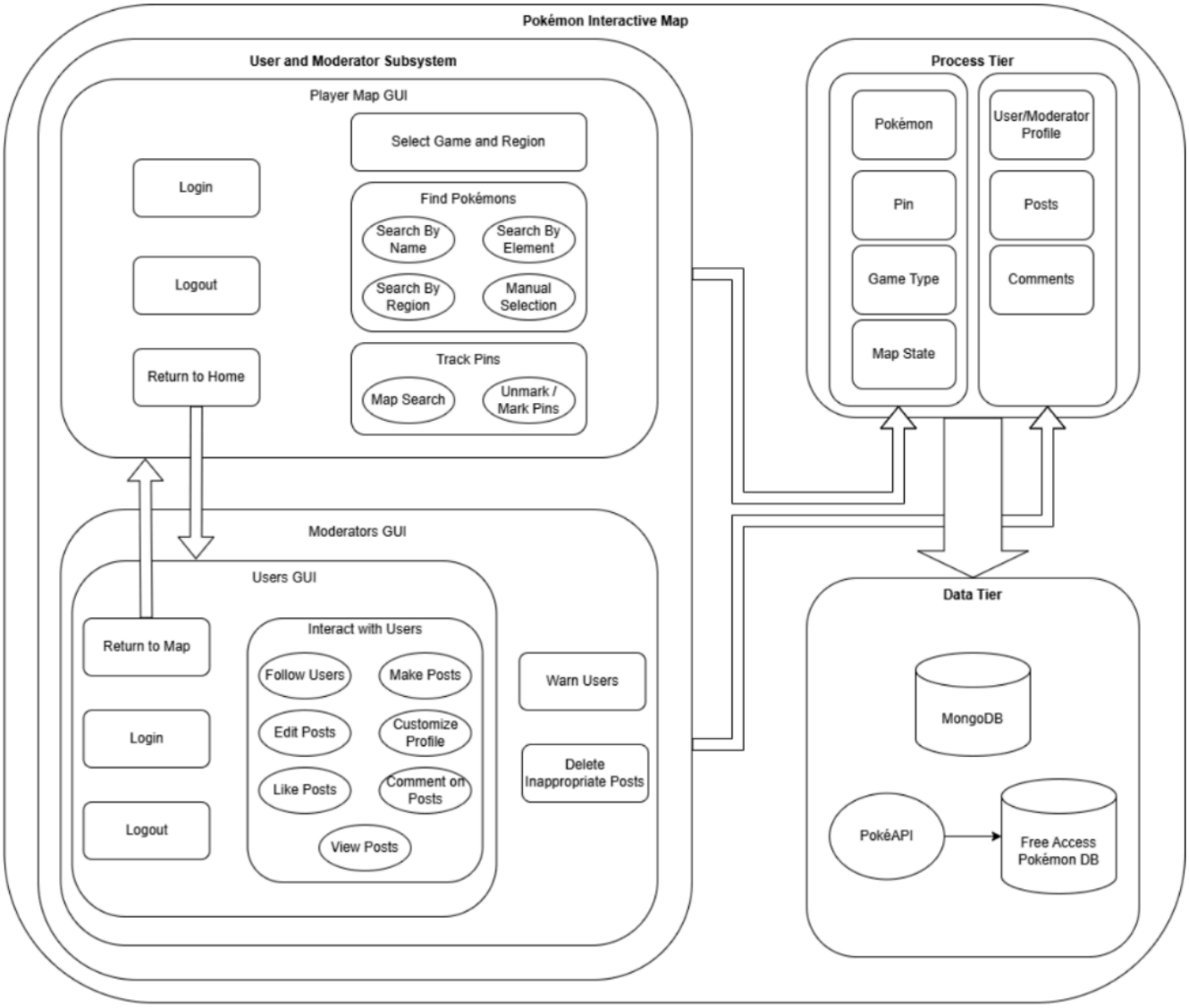
| Dashboard Needs | Functional Requirement | Non-Functional Requirement |
|---|---|---|
| *As a user, I want to customize my own profile.* | Add an editable profile page for each user. | The user icon should take the user to the profile page.<br><br>Customization should include: editing the avatar, the name, the tagline. |
| *As a user, I want to see others' posts and make some of my own.* | Display posts from other users on the dashboard and add a button for the user to create a new post. | Upon loading the dashboard, a list of posts from other users must be shown.<br><br>Each post must have the avatar and name of the original poster along with the time they posted.<br><br>The button to create a new post must be in a fixed location, above the posts list. |

| | | |
|---|---|---|
| *As a user, I want to show other users images of my Pokémons.* | Allow images to be on posts. | Posts must have both text and images format (.png, .jpeg) or any combination of both. |
| *As a user, I want to view and edit or delete my old posts.* | Add a history of old/recent posts of the user in the profile page.<br><br>Add a button on each post for the user to edit.<br><br>Allow the user to delete a post as an option while editing. | Posts must be shown in order of latest to oldest.<br><br>A button in the icon of a pen (for the user to recognize) must be next to a post. Editing any post must update the post's status to being edited and update the time.<br><br>In case a post is deleted, an option to restore the post must appear for roughly 20 seconds should the user want to undo it. |
| *As a user, I want to follow other users and see their posts more regularly.* | Allow the user to select the original posters of any post and follow them.<br><br>Push the posts of followed users to the start of the list. | When selecting any original posters, a button must appear that allows the user to follow them. Once pressed, the system must show that the user is followed within 2 seconds.<br><br>Selecting the poster's avatar must lead to their profile page.<br><br>On the dashboard, all followed users' posts must appear first, then posts from other users. |

| | | |
|---|---|---|
| *As a user, I want to share my opinion about a post.* | Allow the user to like and comment directly under a post. | Upon liking a post, the likes count display must update within 3 seconds, along with a color indicator that the user has liked the post.<br><br>Upon selecting to comment under a post, after typing the comment, the post must display the new comment within 5 seconds. On any post that the user has commented on, their comments will always be shown first. |
| *As a moderator, I want other users to know my status.* | Display the status of a moderator along with regular user profiles. | The moderator tags should be visually different from other profiles (different, vibrant color, bolder font, singular case). |
| *As a moderator, I want to check others' posts and warn them or delete their posts if they are inappropriate.* | Allow the moderator to delete a post on the dashboard or warn the poster. | The buttons to delete a post and warning the poster should be visually different from regular user features (following). Warning a user/deleting a post must also show a form for the moderator to fill in the reasoning. The user that is warned or whose posts are deleted will be shown the reasoning upon login. |
| *As a user, I want to access the map from the dashboard.* | Add an icon for the user to go to the map site. | Loading the map website should take 1 minute or less. The map data should be saved specifically for each user and be resumed upon user login to the map website. |

| Map Needs | Functional Requirement | Non-Functional Requirement |
|---|---|---|
| *As a player, I want to enter the name of a Pokémon.* | Implement a Pokémon Search Bar that accepts user input. | Search should return results within 5 seconds. |
| *As a player, I want to see a list of all game versions (e.g., "Red", "Blue", "Yellow").* | Implement a selector to display a complete list of Pokémon game versions. | The list should be up-to-date. |
| *As a player, I want to find where I could encounter a Pokémon.* | Show all possible locations of that Pokémon on the map as pins. | Pins should be accurate within 5 walking steps from the actual location and should cover all possible locations. |
| *As a player, I want to see the method by which I can find Pokémon.* | Display encounter methods for each Pokémon in a selected location. | For desktop, hovering the cursor over the pin must display the information in 2 seconds.<br><br>For mobile, holding onto the pin must display the information in 2 seconds. |
| *As a player, I want to track my progression of finding Pokémon.* | Enable marking of pins over the map. | Each pin should be able to be marked or unmarked by the user with a response time of 2 seconds. |

### 3.1.2. *Software Architecture*



## 3.2 Hardware

The system's operational requirements are divided into two key areas: the client

and the server.

- **Client Requirements:** The primary requirement for end-users is a standards-compliant modern web browser. The application is designed to be fully accessible from a wide range of devices, including personal computers (PCs), laptops, tablets, and mobile phones.

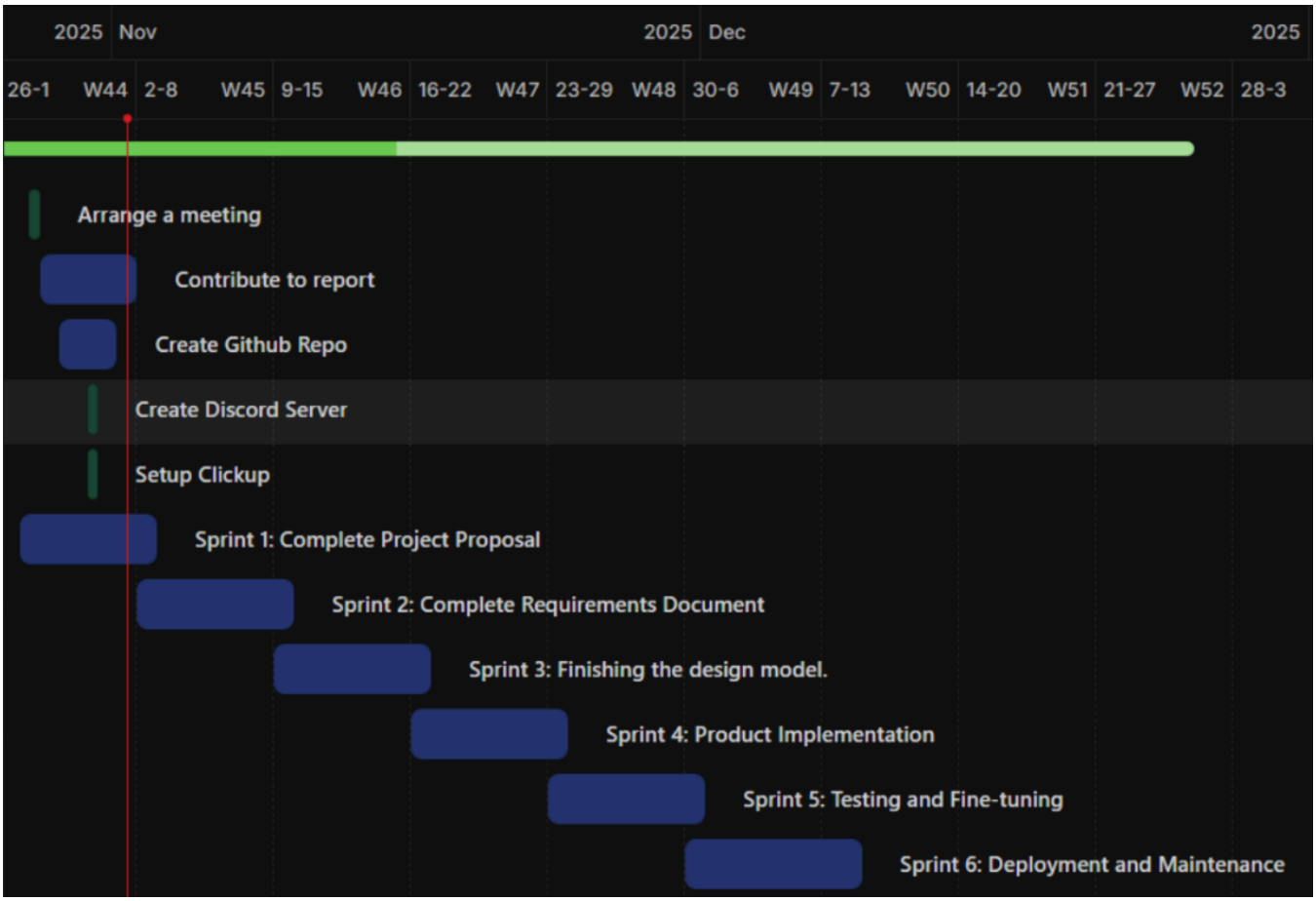
- **Server Requirements:** The application backend is deployed on a dedicated local hosting server to ensure full data control and network security. The server environment must meet the following specifications:

  + *Hardware:* A machine with a multi-core processor (e.g., 4+ cores), a minimum of 8GB RAM (16GB recommended for production loads), and sufficient solid-state drive (SSD) storage to accommodate the application, operating system, and database.

  + *Network:* The server must reside on a stable local area network (LAN) with a static IP address or resolvable hostname. It must have configured firewall rules to allow inbound HTTP (port 80) and HTTPS (port 443) traffic while restricting unauthorized access to other ports.

# 4 Development Plan

Here is the current Gantt chart of our proposed development plan:




## 4.1 *Requirements Analysis*

**Deliverable:** Requirements Specification Document.

**Tasks:**

- *Analyze and detail all Functional and Non-Functional Requirements* for both the Dashboard (user accounts, posting, following, moderation) and the Map (game/region selection, Pokémon search, pin tracking, PokéAPI data display).
- *Analyze the PokéAPI documentation* for the /pokemon/{id-or name}/encounters endpoint to understand the data structure and define the exact location and encounters data to be extracted and stored.
- *Define data models* for MongoDB (user profiles, posts, comments, likes, and user-specific map states like tagged pins and game selection).

## 4.2 Software Design

**Deliverable:** Software Design Document (SDD), UI/UX mockups (in Figma)

**Tasks:**

- *Design the 3-tier architecture:* Defining the ReactJS frontend, Node.js/Express backend API, and MongoDB database layers.
- *Design the full RESTful API contract* between the frontend and the Node.js/Express backend for all features (User Auth, CRUD for Posts/Comments, Following, Moderation, and Map data management).
- *Create comprehensive UI/UX mockups in Figma* for the Dashboard, User Profile, Post creation/viewing, and the interactive Map interface (including the game selector, region list, Pokémon filter/search, and pin display).
- *Design the database schema* for MongoDB.

## 4.3 Implementation

**Deliverable:** Working source code hosted on GitHub.

**Tasks (Phased):**

- *Database Setup (MongoDB)*: Set up the MongoDB instance and define the schemas for all required collections (Users, Posts, Comments, Map States, etc.).
- *Backend Development (Node.js/Express):* Create all API endpoints: Authentication/User Management, CRUD for Posts/Comments/Likes, Following/Moderation Logic, and PokéAPI integration (calling the external API, processing JSON, and caching/storing location data).
- *Frontend Development (ReactJS/Tailwind CSS):* Build the UI components: Navigation, Dashboard, Profile Page, Post Editor/Viewer, and the Interactive Map component with zoom, region selection, and pin functionality.

## 4.4 Testing

**Deliverable:** Test Plan and Test Report.

**Tasks:**

- *Unit Tests:* Perform unit tests on the Node.js backend logic (e.g., MongoDB interactions, user authentication, and PokéAPI JSON processing).
- *Integration Tests (API Testing):* Test all API endpoints using tools like Postman to ensure they correctly handle requests for posting, commenting, liking, and map data updates.
- *System & Acceptance Tests:*
  - Map Functionality Tests: Search for various Pokémon (Common, Rare, No encounter data) to verify pin accuracy and encounter method display.
  - Dashboard/User Tests: Verify profile customization, post creation/editing/deletion, following logic, and moderator actions.

## 4.5 Deployment and Maintenance

**Deliverable:** Deployed web application.

**Tasks:**

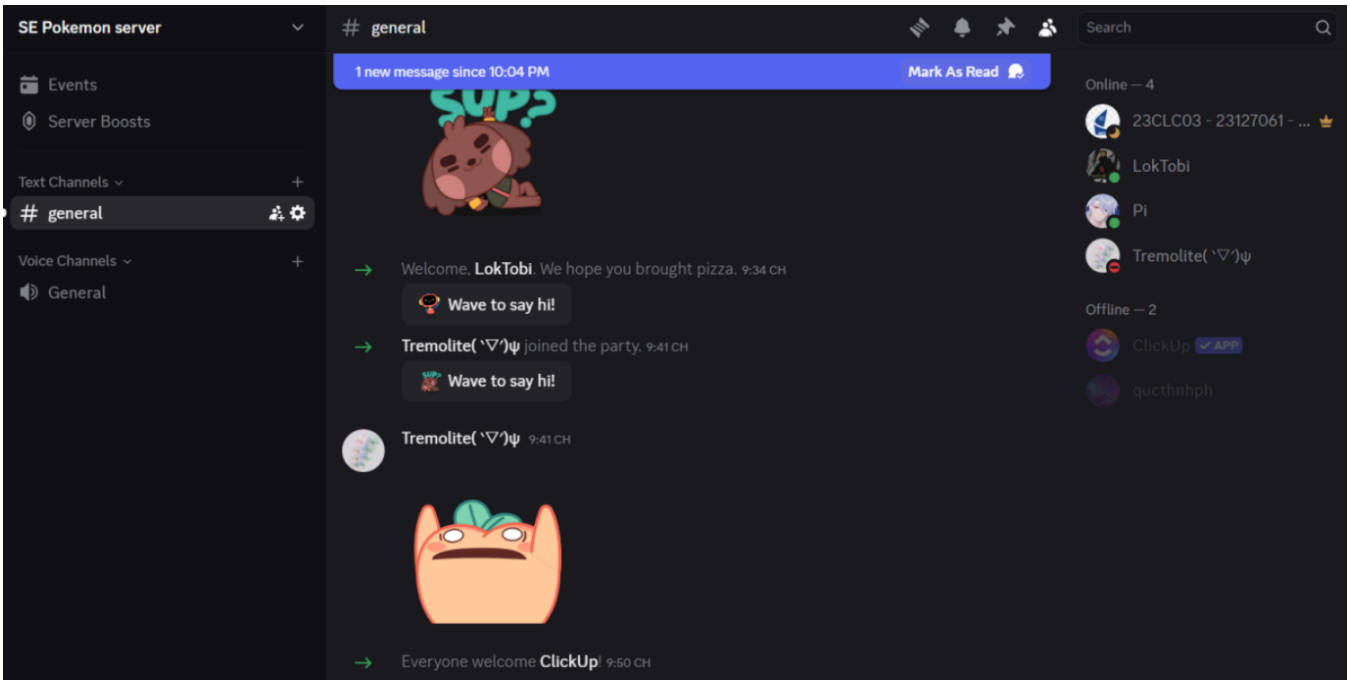
- *Deployment:* Deploy the Node.js/Express application (with the React build) to the local hosting server.
- *Configuration:* Configure firewall rules and ensure the application is accessible via HTTP/HTTPS.
- *Maintenance & Monitoring:* Monitor the application for stability, server resource usage, database performance, and track the status of the external PokéAPI for any potential breaking changes.

# 5 Human Resources & Costing Plan

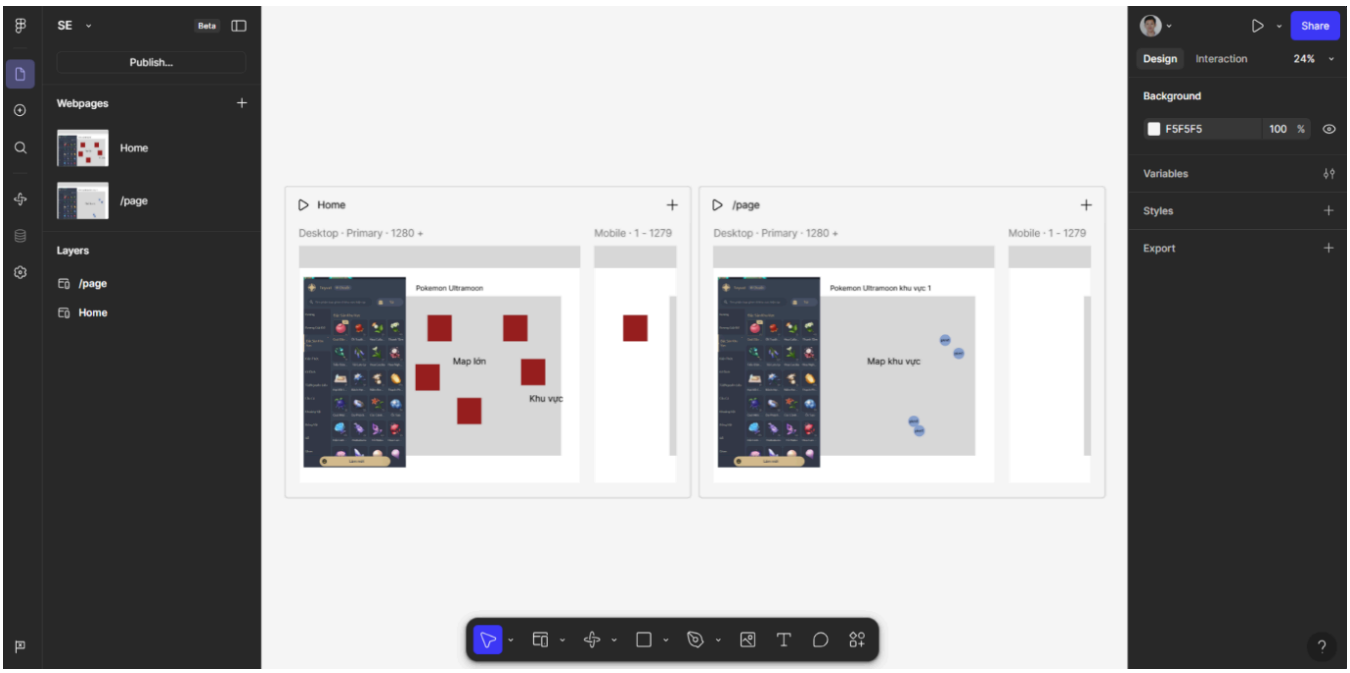
- Accounting for Human Resources, the project team will consist of the 5 student members. For the initial scope, tasks will be divided based on the development plan shown within the previous section.
- The project's total cost is expected to be minimal if not zero. PokéAPI is publicly available and free to use and all relevant maps related to the games are free on the internet. Hosting the website using Apache should also be free and if the data used to store on MongoDB is low enough, there will also be no charges relating to data management.

# 6 Tools setup

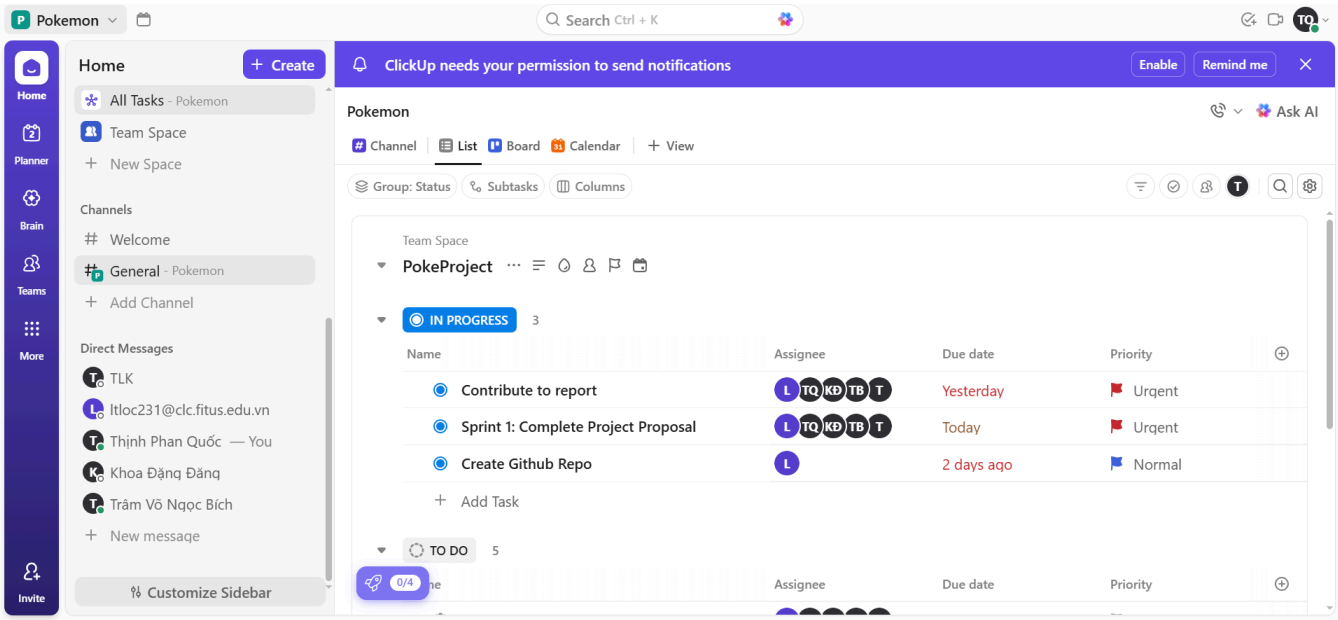
- **Moodle** is used to manage all direct assignments and submissions. All members will access Moodle on PC and only one member will represent the group when submitting the project.
- **Discord** is where team members will arrange and host meetings and task collaborations, as well as general discussions and messaging.



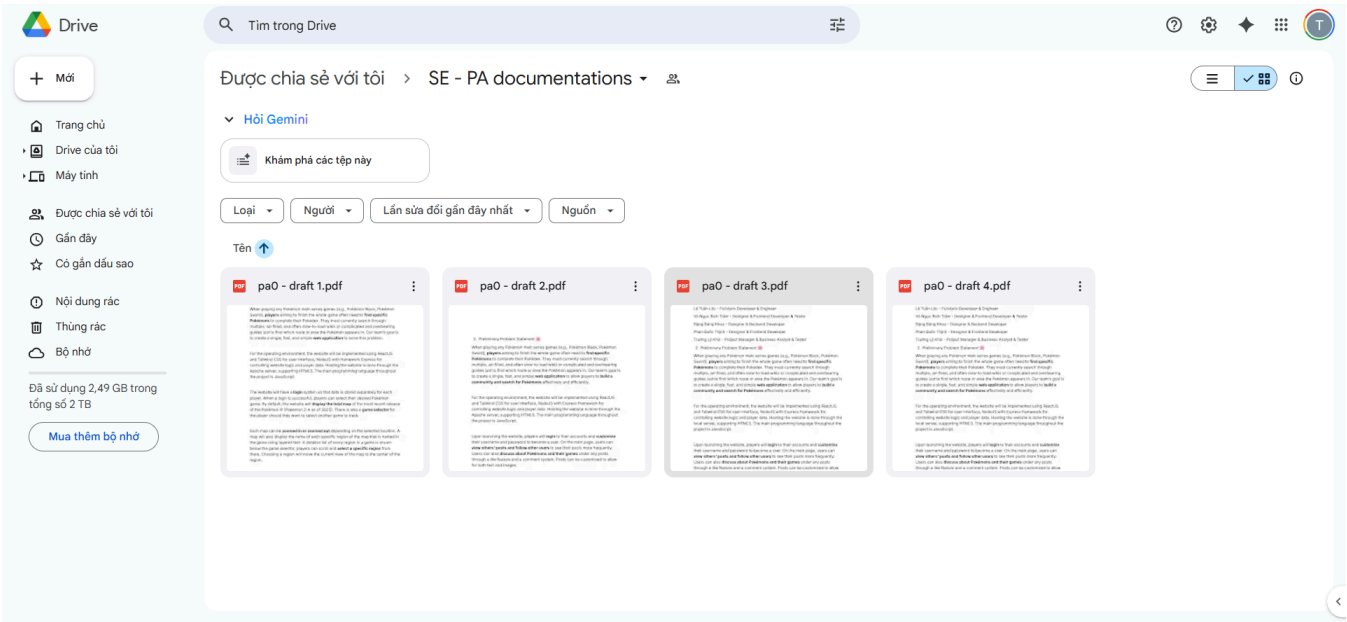
- **Figma** is used to handle general design of the web in an easy way.

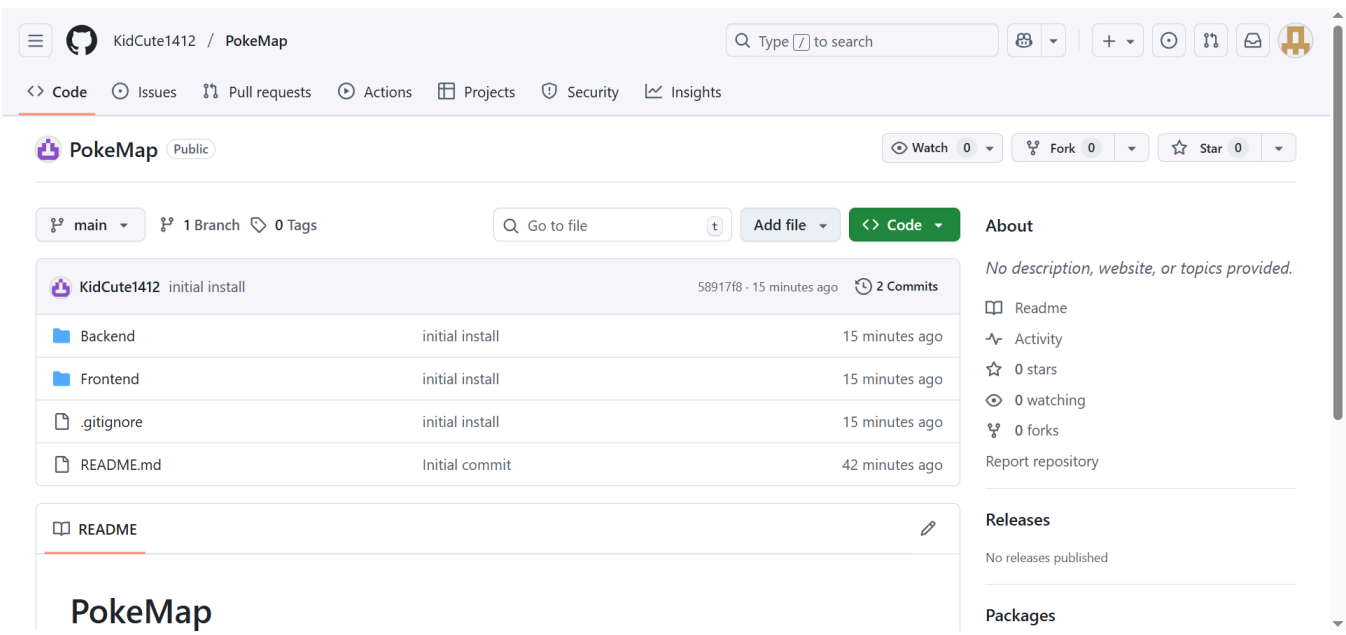



- **Clickup** will handle tasks management for individual members, these tasksonce assigned will send a notification to the discord channel for everyone to see. Clickup is also used to create the Gantt chart for our project.



- **Google Drive** handles important files and reports that are ready for submissions.




- **GitHub** is used for source control and documentation for the project.